

Guia Técnico Completo — MV Energia Solar

Mentor de Arquitetura: FastAPI + SQLAlchemy + Streamlit

Para quem é este guia: Desenvolvedor com sólido background em VBA/Excel/Access, conhecimento básico de Python, migrando para desenvolvimento web profissional com Python moderno.

Índice

1. [Visão Geral da Arquitetura](#)
2. [Estrutura de Pastas e Modularização](#)
3. [Banco de Dados com SQLAlchemy](#)
4. [Migrations com Alembic](#)
5. [Validação com Pydantic](#)
6. [FastAPI: Rotas, Dependências e Organização](#)
7. [Camada de Serviços \(Services\)](#)
8. [Autenticação Simples](#)
9. [Streamlit como Frontend](#)
10. [Deploy na Railway e Streamlit Cloud](#)
11. [Boas Práticas que este Projeto Aplica](#)
12. [Pontos de Atenção e o que Fazer Diferente](#)

1. Visão Geral da Arquitetura

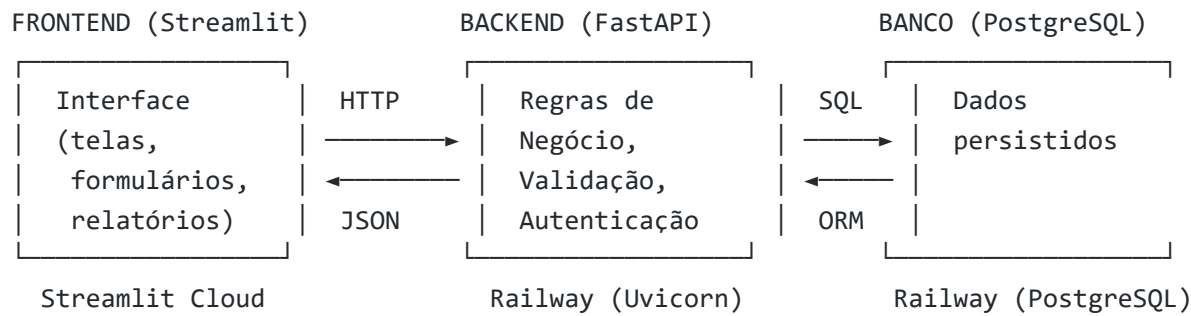
O problema que a separação resolve

Em VBA, tudo vive junto: o botão do formulário chama diretamente o módulo, que abre uma conexão ADODB, executa o SQL e devolve o resultado para o formulário — tudo no mesmo arquivo `.xlsm` ou `.accdb`. Funciona para uso local de uma pessoa ou de um escritório pequeno.

O problema começa quando:

- Mais de uma pessoa precisa usar ao mesmo tempo
- O sistema precisa rodar em qualquer dispositivo (celular, outro computador)
- Você quer trocar a interface sem mexer nas regras de negócio
- Precisa escalar (mais usuários, mais dados)

A solução moderna separa em três camadas independentes:



Analogia direta com o que você já conhece

Camada Moderna	Equivalente VBA/Access
Streamlit (Frontend)	Formulário UserForm do Excel
FastAPI (Backend)	Módulo VBA com as Sub/Function
PostgreSQL (Banco)	Tabelas do Access / planilha Excel
HTTP/JSON	Chamada de função entre módulos
Pydantic Schema	Type checking do parâmetro de função
SQLAlchemy Model	Definição de tabela no Access
Alembic Migration	ALTER TABLE que você fazia na mão

A diferença fundamental: no VBA, o formulário e o módulo estão no mesmo processo. Aqui, Streamlit e FastAPI são **processos separados em servidores diferentes**, e eles se falam via HTTP (o mesmo protocolo do navegador web).

Por que HTTP + JSON?

Quando o Streamlit precisa listar clientes, ele faz:

```
response = requests.get("https://mvenergiasolar-production.up.railway.app/clientes/")
clientes = response.json() # Lista de dicionários Python
```

É como se o formulário Excel ligasse para um serviço externo dizendo: "me dá a lista de clientes em formato que eu entendo" — e recebe de volta um JSON (que é praticamente um dicionário Python / objeto JavaScript).

Por que FastAPI + SQLAlchemy + Python 3.13?

- **FastAPI:** o framework mais rápido para construir APIs em Python. Usa as anotações de tipo do Python moderno para fazer validação automática e gerar documentação interativa. Simples de aprender, mas poderoso o suficiente para produção.
- **SQLAlchemy:** ORM maduro (existe desde 2006), funciona com SQLite (desenvolvimento) e PostgreSQL (produção) com zero mudança de código.
- **Python 3.13:** versão atual com melhorias de performance e sintaxe limpa. As anotações de tipo (`str` , `int` , `Optional[str]`) são fundamentais para o FastAPI funcionar.

Ponto-chave: A separação frontend/backend/banco não é burocracia — é o que permite que o sistema funcione em múltiplos dispositivos simultaneamente, seja mantido por pessoas diferentes e escale sem reescrever tudo. A analogia com VBA é válida: você já conhece as peças, agora elas só ficam em lugares diferentes e conversam via HTTP.

2. Estrutura de Pastas e Modularização

A estrutura ideal para este projeto

```

mv_energia_solar/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   └── database.py
│   ├── models/
│   │   ├── __init__.py
│   │   ├── cliente.py
│   │   ├── produto.py
│   │   ├── venda.py
│   │   ├── colaborador.py
│   │   ├── estoque.py
│   │   ├── financeiro.py
│   │   ├── prestacao.py
│   │   ├── projeto.py
│   │   ├── ordem_servico.py
│   │   └── usuario.py
│   └── schemas/
│       ├── __init__.py
│       ├── cliente.py
│       ├── produto.py
│       ├── venda.py
│       └── ...
└── routers/

```

Tudo do FastAPI

Ponto de entrada: cria o app FastAPI
Configuração da conexão com o banco

Definição das tabelas (SQLAlchemy)

Validação de entrada/saída (Pydantic)

Endpoints HTTP por domínio

```

├── __init__.py
├── clientes.py
├── produtos.py
├── vendas.py
├── estoque.py
├── financeiro.py
├── prestacoes.py
├── colaboradores.py
├── projetos.py
├── ordens_servico.py
├── usuarios.py
├── dashboard.py
├── services/                # Lógica de negócio complexa
│   ├── __init__.py
│   ├── venda_service.py
│   ├── estoque_service.py
│   └── financeiro_service.py
├── migrations/              # Gerado pelo Alembic automaticamente
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│       ├── 001_create_clientes.py
│       └── 002_add_campo_xyz.py
├── alembic.ini              # Configuração do Alembic
├── requirements.txt         # Dependências Python
├── .env                     # Variáveis de ambiente (NÃO versionar)
├── .env.example             # Template das variáveis (versionar)
├── frontend/                # Tudo do Streamlit
│   ├── app.py               # Login + Dashboard
│   └── pages/
│       ├── 01_Clientes.py
│       ├── 02_Vendas.py
│       ├── 03_Estoque.py
│       ├── 04_Financeiro.py
│       ├── 05_Prestacoes.py
│       ├── 06_Colaboradores.py
│       ├── 07_Projetos.py
│       ├── 08_Manutencao.py
│       ├── 09_Usuarios.py
│       ├── 10_Ficha_Cliente.py
│       ├── 11_Prospeccao.py
│       ├── 12_Relatorios.py
│       ├── 13_Importacao.py
│       └── 14_Previsao.py
│   ├── utils/
│       ├── api.py           # Funções para chamar o backend
│       └── auth.py          # Helpers de autenticação
├── requirements.txt

```

|
└─ README.md

O papel de cada pasta

models/ — É onde você descreve como o banco de dados está estruturado, em Python. Cada arquivo é uma tabela. Pense como a definição de uma tabela no Access Designer — mas em código. Uma vez escrito, o Alembic cria (e atualiza) a tabela real por você.

schemas/ — São os "contratos" de entrada e saída da API. Quando alguém chama `POST /clientes/`, o schema define exatamente quais campos são obrigatórios, quais são opcionais, e como validá-los. Isso é separado do model porque o que você aceita na entrada pode ser diferente do que você guarda no banco (e diferente do que você devolve na saída).

routers/ — São os "endereços" da API organizados por assunto. Em vez de colocar todos os endpoints em um arquivo gigante, você separa: tudo sobre clientes fica em `routers/clientes.py`, tudo sobre vendas em `routers/vendas.py`, etc. É como ter um módulo VBA por entidade.

services/ — Quando uma operação envolve múltiplas tabelas, regras complexas ou transações, você não coloca isso dentro do router. Você cria um serviço. Exemplo: aprovar uma venda (que baixa estoque + cria lançamento financeiro + gera prestação de contas) fica em `venda_service.py`.

database.py — Configura a conexão com o banco. Uma vez escrito, você raramente toca aqui.

Módulo em Python vs Módulo em VBA

Em VBA, um módulo é um arquivo `.bas` dentro do projeto. Em Python, **qualquer arquivo `.py` é um módulo**. A diferença chave é que em Python você importa explicitamente o que precisa:

```
# VBA (implícito – tudo está acessível no projeto)
Call CalcularComissao(venda_id)

# Python (explícito – você diz de onde vem)
from app.services.venda_service import aprovar_venda
resultado = aprovar_venda(db, venda_id)
```

O arquivo `__init__.py` (vazio ou com imports) diz ao Python "esta pasta é um pacote importável". É o equivalente de registrar um módulo.

Ponto-chave: A estrutura de pastas não é estética — ela é o que torna o projeto navegável e mantível. Cada pasta tem uma responsabilidade única: `models` falam com o banco, `schemas` validam dados, `routers` recebem requisições, `services` contêm lógica de negócio. Quando algo quebra, você sabe exatamente onde olhar.

3. Banco de Dados com SQLAlchemy

O que é um ORM e por que usar

ORM significa *Object-Relational Mapper*. Em vez de escrever SQL na mão, você define classes Python e o ORM traduz tudo para SQL automaticamente.

Em VBA + Access, você provavelmente escrevia algo como:

```
' VBA com ADODB
Dim rs As ADODB.Recordset
rs.Open "SELECT * FROM clientes WHERE status = 'Ativo'", conn
```

Com SQLAlchemy, a mesma operação fica:

```
# Python com SQLAlchemy
clientes = db.query(Cliente).filter(Cliente.status == StatusCliente.ativo).all()
```

A vantagem vai além da sintaxe: o ORM sabe que em SQLite o tipo `Boolean` é `INTEGER(1)` e em PostgreSQL é `BOOLEAN`. Você escreve uma vez e funciona nos dois.

Configuração da conexão (`database.py`)

```
# app/database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

# Usa a variável de ambiente; cai para SQLite se não existir (desenvolvimento)
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./solar.db")

# PostgreSQL na Railway usa "postgres://" mas SQLAlchemy exige "postgresql://"
if DATABASE_URL.startswith("postgres://"):
    DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)

engine = create_engine(
    DATABASE_URL,
    # Necessário APENAS para SQLite (que não suporta múltiplas threads)
    connect_args={"check_same_thread": False} if "sqlite" in DATABASE_URL else {}
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Classe base da qual todos os models herdam
Base = declarative_base()
```

Analogia: O `engine` é a string de conexão ADODB. O `SessionLocal` é o equivalente a abrir um `Recordset` — mas ao invés de abrir uma vez globalmente, você abre e fecha uma sessão por requisição HTTP.

Declarando Models (tabelas) — exemplos reais do projeto

Enums — definindo valores permitidos

```
# app/models/enums.py
import enum

class StatusCliente(str, enum.Enum):
    ativo      = "Ativo"
    inativo    = "Inativo"
    prospecto  = "Prospecto"
    bloqueado  = "Bloqueado"

class TipoPessoa(str, enum.Enum):
    pf = "PF"
    pj = "PJ"

class StatusVenda(str, enum.Enum):
    orcamento      = "Orçamento"
    negociacao      = "Negociação"
    aprovado        = "Aprovado"
    em_andamento   = "Em Andamento"
    concluido        = "Concluído"
    cancelado        = "Cancelado"

class FormaPagamento(str, enum.Enum):
    pix              = "PIX"
    boleto           = "Boleto"
    dinheiro          = "Dinheiro"
    cartao            = "Cartão"
    financiamento    = "Financiamento"

# ... defina todos os enums do JSON aqui
```

O `str, enum.Enum` é importante: herdar de `str` garante que o valor é serializado como texto (não como `"StatusCliente.ativo"`), o que o FastAPI precisa para JSON.

Analogia com VBA: É como uma Validation Rule numa coluna do Access, mas tipada — Python impede que você atribua um valor inválido em tempo de execução.

Model de Cliente

```
# app/models/cliente.py
from sqlalchemy import Column, Integer, String, Date, Numeric, Enum as SAEnum
from app.database import Base
```

```

from app.models.enums import StatusCliente, TipoPessoa, Escolaridade, TipoImovel, OrigemLead

class Cliente(Base):
    __tablename__ = "clientes"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    status = Column(SAEnum(StatusCliente), nullable=False, default=StatusCliente.ativo)
    tipo_pessoa = Column(SAEnum(TipoPessoa), nullable=False, default=TipoPessoa.pf)
    nome = Column(String(150), nullable=False)
    cpf_cnpj = Column(String(14), nullable=True, unique=True)
    email = Column(String(100), nullable=True)
    telefone = Column(String(20), nullable=True)
    cidade = Column(String(100), nullable=True)
    uf = Column(String(2), nullable=True)
    data_cadastro = Column(Date, nullable=False)
    profissao = Column(String(100), nullable=True)
    renda_mensal = Column(Numeric(12, 2), nullable=True)
    escolaridade = Column(SAEnum(Escolaridade), nullable=True)
    conta_energia = Column(Numeric(10, 2), nullable=True)
    tipo_imovel = Column(SAEnum(TipoImovel), nullable=True)
    origem_lead = Column(SAEnum(OrigemLead), nullable=True)
    endereco = Column(String(200), nullable=True)
    bairro = Column(String(100), nullable=True)

    # Relacionamentos (veja seção abaixo)
    vendas = relationship("Venda", back_populates="cliente")
    projetos = relationship("ProjetoHomologacao", back_populates="cliente")

```

Model de Produto com propriedade calculada

```

# app/models/produto.py
from sqlalchemy import Column, Integer, String, Numeric, Enum as SAEnum
from sqlalchemy.orm import relationship
from app.database import Base
from app.models.enums import CategoriaProduto, UnidadeProduto

class Produto(Base):
    __tablename__ = "produtos"

    id = Column(Integer, primary_key=True, autoincrement=True)
    codigo = Column(String(20), nullable=False, unique=True)
    nome = Column(String(150), nullable=False)
    categoria = Column(SAEnum(CategoriaProduto), nullable=False)
    unidade = Column(SAEnum(UnidadeProduto), nullable=False)
    fabricante = Column(String(100), nullable=True)
    modelo = Column(String(100), nullable=True)
    potencia_w = Column(Numeric(10, 2), nullable=True)
    custo_unitario = Column(Numeric(10, 2), nullable=False)
    preco_venda = Column(Numeric(10, 2), nullable=False)
    qtd_minima = Column(Integer, nullable=False, default=0)

```

```

movimentacoes = relationship("EstoqueMovimentacao", back_populates="produto")

# Propriedade calculada: NÃO vai para o banco.
# É calculada em Python toda vez que você acessa o objeto.
@property
def margem_pct(self) -> float:
    if self.preco_venda and self.preco_venda > 0:
        return float((self.preco_venda - self.custo_unitario) / self.preco_venda)
    return 0.0

```

Por que NÃO guardar campos calculados no banco

Esta é uma das decisões de design mais importantes do projeto. `margem_pct`, `qtd_estoque` e `subtotal` **não existem como colunas** — eles são calculados sob demanda. Por quê?

1. **Consistência:** Se você guardasse `margem_pct = 0.30` e depois alguém atualizasse o `preco_venda`, a margem ficaria desatualizada. Com `@property`, o cálculo é sempre feito com os valores atuais.
2. **Simplicidade:** Menos colunas = menos migrations = menos risco de dados inconsistentes.
3. **Banco de dados deve guardar fatos, não derivações:** O princípio é que o banco guarda dados *primários* (custo, preço), e a lógica deriva o resto.

O `qtd_estoque` é um caso especial — ele não pode ser um `@property` simples porque depende de uma query no banco (soma de todas as movimentações). Por isso é calculado na query:

```

# Em estoque_service.py
from sqlalchemy import func

def calcular_saldo(db: Session, produto_id: int) -> float:
    # Soma as quantidades com sinal (+Entrada, -Saída)
    resultado = db.query(
        func.sum(
            case(
                (EstoqueMovimentacao.tipo_mov.in_(["Entrada", "Ajuste+" ]),
                EstoqueMovimentacao.quantidade),
                else_=-EstoqueMovimentacao.quantidade
            )
        )
    ).filter(EstoqueMovimentacao.id_produto == produto_id).scalar()
    return float(resultado or 0)

```

Analogia VBA: É como uma coluna calculada em uma query do Access. Você não guarda o resultado na tabela — você calcula na hora da consulta.

PKs manuais vs autoincrement

O projeto usa os dois, intencionalmente:

PK autoincrement (Integer) – para entidades onde o ID não tem significado de negócio

```
class Cliente(Base):
    id = Column(Integer, primary_key=True, autoincrement=True)
    # ID gerado: 1, 2, 3, 4...
```

PK manual (String) – para entidades onde o ID tem significado operacional

```
class Colaborador(Base):
    id_colaborador = Column(String(6), primary_key=True)
    # ID definido pelo usuário: "COL001", "COL002"
```

```
class OrdemServico(Base):
    id_os = Column(String(6), primary_key=True)
    # ID definido pelo usuário: "OS0001", "OS0002"
```

Por que PKs manuais? Um colaborador "COL001" aparece em documentos físicos, ordens de serviço impressas, e-mails. O código faz sentido para os humanos que usam o sistema. Um cliente com ID "47" não tem esse valor semântico — ninguém vai mencionar "o cliente 47" em uma conversa.

Desvantagem: Você precisa garantir unicidade manualmente no código (gerar o próximo ID antes de inserir). Em sistemas enterprise, isso causaria race conditions com múltiplos usuários simultâneos — usaríamos UUID ou sequence do banco.

Relacionamentos com relationship

```
# app/models/venda.py
from sqlalchemy import Column, Integer, String, Date, Numeric, Text, Enum as SAEnum, ForeignKey
from sqlalchemy.orm import relationship
from app.database import Base
from app.models.enums import StatusVenda, FormaPagamento

class Venda(Base):
    __tablename__ = "vendas"

    id = Column(Integer, primary_key=True, autoincrement=True)
    cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
    id_responsavel = Column(String(6), ForeignKey("colaboradores.id_colaborador"), nullable=False)
    data_orcamento = Column(Date, nullable=False)
    data_aprovacao = Column(Date, nullable=True)
    status_venda = Column(SAEnum(StatusVenda), nullable=False, default=StatusVenda.orcamen)
    valor_instalacao = Column(Numeric(10, 2), default=0)
    desconto = Column(Numeric(10, 2), default=0)
    forma_pagamento = Column(SAEnum(FormaPagamento), nullable=False)
    parcelas = Column(Integer, default=1)
    observacoes = Column(Text, nullable=True)

    # Relacionamentos – permitem acessar objetos relacionados
    cliente = relationship("Cliente", back_populates="vendas")
    itens = relationship("ItemVenda", back_populates="venda", cascade="all, delete-")
    responsavel = relationship("Colaborador", back_populates="vendas")
```

```
lancamentos = relationship("Financeiro", back_populates="venda")
```

```
@property
```

```
def valor_equipamentos(self) -> float:
```

```
    return sum(float(item.quantidade * item.preco_unitario) for item in self.itens)
```

```
@property
```

```
def valor_final(self) -> float:
```

```
    return self.valor_equipamentos + float(self.valor_instalacao or 0) - float(self.descon
```

Como o `relationship` funciona: Quando você faz `venda.cliente`, o SQLAlchemy executa automaticamente um `SELECT` na tabela `clientes` para trazer o objeto completo. Isso é o **lazy loading** — só carrega quando você acessa. Para evitar N+1 queries em listas, usa-se `joinedload`:

```
# Carrega vendas já com o cliente em uma única query (eager loading)
vendas = db.query(Venda).options(joinedload(Venda.cliente)).all()
```

Analogia VBA: É como um JOIN do Access, mas automático. Em vez de escrever `SELECT v.*, c.nome FROM vendas v JOIN clientes c ON v.cliente_id = c.id`, você acessa `venda.cliente.nome` e o ORM faz o JOIN por você.

Ponto-chave: Models descrevem a estrutura do banco em Python. Campos calculados (`@property`) nunca são colunas — são derivados em tempo de execução. PKs manuais (String) fazem sentido quando o código tem significado operacional. `relationship` é um JOIN automático lazy — use `joinedload` para evitar queries em excesso em listas.

4. Migrations com Alembic

O problema que o Alembic resolve

Imagine que você lançou a versão 1.0 do sistema. O banco tem as tabelas. Três meses depois, precisa adicionar a coluna `escolaridade` na tabela de clientes.

Sem Alembic, você entraria no banco e executaria:

```
ALTER TABLE clientes ADD COLUMN escolaridade VARCHAR(50);
```

Mas e se tiver 5 ambientes (desenvolvimento local, staging, produção)? E se outro desenvolvedor entrar? E se precisar desfazer? Você tem que lembrar de tudo manualmente.

Com Alembic, você cria um arquivo de migration que descreve a mudança, e o comando `alembic upgrade head` aplica todas as mudanças pendentes automaticamente, em qualquer ambiente.

Analogia VBA: É como ter um script de versão para o banco do Access, mas automático e versionado no Git.

Fluxo completo

1. Configuração inicial (uma vez)

```
# Instala alembic
pip install alembic

# Inicializa na pasta do projeto
alembic init migrations

# alembic.ini é criado – edite a linha sqlalchemy.url:
# sqlalchemy.url = sqlite:///./solar.db
# (ou use a variável de ambiente, melhor prática)
```

2. Configurar migrations/env.py para reconhecer seus models

```
# migrations/env.py – seção importante
from app.database import Base
from app.models import cliente, produto, venda, colaborador # importe todos

target_metadata = Base.metadata
```

3. Criar uma migration (depois de alterar/criar um model)

```
# Alembic compara seus models com o estado atual do banco e gera o script
alembic revision --autogenerate -m "cria_tabela_clientes"
```

Isso gera um arquivo em migrations/versions/ parecido com:

```
# migrations/versions/001_cria_tabela_clientes.py
def upgrade() -> None:
    op.create_table(
        'clientes',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('nome', sa.String(150), nullable=False),
        sa.Column('status', sa.Enum('Ativo', 'Inativo', 'Prospecto', 'Bloqueado',
                                    name='statuscliente'), nullable=False),

        # ... outros campos
        sa.PrimaryKeyConstraint('id')
    )
```

```
def downgrade() -> None:
    op.drop_table('clientes')
```

SEMPRE revise este arquivo antes de aplicar. O `--autogenerate` é bom, mas não é perfeito. Ele pode não detectar renomeações de colunas corretamente (detecta como drop+add).

4. Aplicar a migration

```
alembic upgrade head      # Aplica todas as pendentes
alembic upgrade +1        # Aplica só a próxima
alembic downgrade -1      # Desfaz a última
alembic current           # Mostra qual versão está aplicada
alembic history           # Lista todas as migrations
```

O comando de start na Railway

```
alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

O `&&` significa: "execute o segundo comando APENAS se o primeiro tiver sucesso". Então:

1. Quando o container sobe na Railway, primeiro roda `alembic upgrade head`
2. Isso aplica todas as migrations pendentes no banco PostgreSQL
3. **Só então** sobe o servidor Uvicorn para receber requisições

Isso garante que o banco sempre está na versão correta antes do backend aceitar tráfego. É elegante e seguro.

Ponto-chave: Migration é um log versionado de todas as mudanças no banco. Você nunca mais vai "alterar a tabela na mão" — você cria uma migration, comita no Git, e o deploy aplica automaticamente. O `alembic upgrade head` no start command é o que sincroniza o banco com o código a cada deploy.

5. Validação com Pydantic

O que são schemas e por que existem separados dos models

Esta é uma das partes que mais confunde quem está começando: **model** $\neq$ **schema**.

- **Model** (SQLAlchemy): descreve como os dados são *guardados no banco*. Tem colunas, PKs, FKs, índices.
- **Schema** (Pydantic): descreve como os dados *entram e saem da API*. Tem validações, campos obrigatórios, formatação.

Por que separar? Porque são responsabilidades diferentes:

Situação	O que acontece
POST /clientes/	Você recebe um JSON, o schema <code>ClienteCreate</code> valida os campos antes de qualquer coisa chegar ao banco
GET /clientes/	Você retorna objetos do banco, o schema <code>ClienteResponse</code> define exatamente quais campos aparecem no JSON
PATCH /clientes/{id}	Você recebe apenas os campos a atualizar, o schema <code>ClienteUpdate</code> tem todos os campos como <code>Optional</code>

Os três tipos de schema — exemplo com Cliente

```
# app/schemas/cliente.py
from pydantic import BaseModel, field_validator
from typing import Optional
from datetime import date
from app.models.enums import StatusCliente, TipoPessoa, Escolaridade, TipoImovel, OrigemLead

# =====
# Schema de CRIAÇÃO – campos que o usuário envia no POST
# =====
class ClienteCreate(BaseModel):
    nome: str # Obrigatório
    tipo_pessoa: TipoPessoa = TipoPessoa.pf # Opcional com default
    status: StatusCliente = StatusCliente.ativo
    cpf_cnpj: Optional[str] = None
    email: Optional[str] = None
    telefone: Optional[str] = None
    cidade: Optional[str] = None
    uf: Optional[str] = None
    data_cadastro: date
    profissao: Optional[str] = None
    renda_mensal: Optional[float] = None
    escolaridade: Optional[Escolaridade] = None
    conta_energia: Optional[float] = None
    tipo_imovel: Optional[TipoImovel] = None
    origem_lead: Optional[OrigemLead] = None
    endereco: Optional[str] = None
    bairro: Optional[str] = None

    # Validação customizada de CPF/CNPJ
    @field_validator("cpf_cnpj", mode="before")
    @classmethod
    def validar_cpf_cnpj(cls, v: Optional[str]) -> Optional[str]:
        if v is None or v.strip() == "":
            return None
```

```

# Remove tudo que não for dígito
digitos = "".join(c for c in v if c.isdigit())
if len(digitos) not in (11, 14):
    raise ValueError("CPF deve ter 11 dígitos ou CNPJ 14 dígitos")
return digitos # Armazena só os dígitos

# =====
# Schema de ATUALIZAÇÃO – todos os campos são opcionais (PATCH)
# =====
class ClienteUpdate(BaseModel):
    nome: Optional[str] = None
    status: Optional[StatusCliente] = None
    tipo_pessoa: Optional[TipoPessoa] = None
    cpf_cnpj: Optional[str] = None
    email: Optional[str] = None
    telefone: Optional[str] = None
    cidade: Optional[str] = None
    uf: Optional[str] = None
    # ... todos os campos como Optional

    @field_validator("cpf_cnpj", mode="before")
    @classmethod
    def validar_cpf_cnpj(cls, v):
        # Mesma lógica do Create
        if v is None or v.strip() == "":
            return None
        digitos = "".join(c for c in v if c.isdigit())
        if len(digitos) not in (11, 14):
            raise ValueError("CPF deve ter 11 dígitos ou CNPJ 14 dígitos")
        return digitos

# =====
# Schema de RESPOSTA – o que a API devolve no GET
# =====
class ClienteResponse(BaseModel):
    id: int
    nome: str
    status: StatusCliente
    tipo_pessoa: TipoPessoa
    cpf_cnpj: Optional[str] = None
    email: Optional[str] = None
    telefone: Optional[str] = None
    cidade: Optional[str] = None
    uf: Optional[str] = None
    data_cadastro: date
    # Campos sensíveis como renda_mensal podem ser omitidos se necessário

    model_config = {"from_attributes": True}
    # Isso permite criar o schema a partir de um objeto SQLAlchemy:
    # ClienteResponse.model_validate(cliente_do_banco)

```

O `from_attributes = True` — detalhe crítico

Sem essa configuração, o Pydantic só aceita dicionários como entrada. Com ela, aceita objetos Python (incluindo os objetos do SQLAlchemy). Você verá isso em **todos** os schemas de Response.

Como o FastAPI usa Pydantic automaticamente

```
@router.post("/clientes/", response_model=ClienteResponse)
def criar_cliente(dados: ClienteCreate, db: Session = Depends(get_db)):
    # O FastAPI já validou `dados` antes de entrar aqui.
    # Se `nome` estivesse ausente, o FastAPI retornaria 422 automaticamente.
    # Se `cpf_cnpj` fosse "abc123", o validator rodaria e retornaria 422.
    pass
```

Analogia VBA: É como ter um `If IsNull(Me.txtNome) Then MsgBox "Nome é obrigatório"` em cada formulário — mas automático, para todos os campos, com mensagens de erro padronizadas em JSON.

Ponto-chave: Separe sempre Create/Update/Response. O schema de Create define o contrato de entrada, o Response define o contrato de saída, e o Update tem tudo como Optional para suportar PATCH (atualização parcial). O validador de CPF/CNPJ mostra como adicionar regras de negócio na validação sem poluir o router ou o service.

6. FastAPI: Rotas, Dependências e Organização

O `main.py` — ponto de entrada

```
# app/main.py
from fastapi import FastAPI
from app.routers import clientes, produtos, vendas, colaboradores
from app.routers import estoque, financeiro, prestacoes, projetos
from app.routers import ordens_servico, usuarios, dashboard
from app.database import Base, engine
from app import models # Importa todos os models para o Base conhecer

app = FastAPI(
    title="MV Energia Solar API",
    version="1.0",
    description="Sistema de gestão para empresa de energia solar fotovoltaica"
)

# Registra cada router com seu prefixo
app.include_router(clientes.router, prefix="/clientes", tags=["Clientes"])
app.include_router(produtos.router, prefix="/produtos", tags=["Produtos"])
app.include_router(vendas.router, prefix="/vendas", tags=["Vendas"])
```

```

app.include_router(colaboradores.router, prefix="/colaboradores", tags=["Colaboradores"])
app.include_router(estoque.router, prefix="/estoque", tags=["Estoque"])
app.include_router(financeiro.router, prefix="/financeiro", tags=["Financeiro"])
app.include_router(prestacoes.router, prefix="/prestacoes", tags=["Prestações"])
app.include_router(projetos.router, prefix="/projetos", tags=["Projetos"])
app.include_router(ordens_servico.router, prefix="/ordens-servico", tags=["Ordens de Serviço"])
app.include_router(usuarios.router, prefix="/usuarios", tags=["Usuários"])
app.include_router/dashboard.router, prefix="/dashboard", tags=["Dashboard"])

# Cria as tabelas se não existirem (útil em desenvolvimento; em produção use Alembic)
@app.on_event("startup")
def on_startup():
    Base.metadata.create_all(bind=engine)
    _criar_admin_se_necessario()

```

Injeção de Dependência com Depends(get_db)

Este é o padrão mais importante do FastAPI para banco de dados. A ideia é: cada requisição HTTP recebe sua própria sessão de banco de dados, que é fechada (e o commit feito ou rollback executado) ao final.

```

# app/database.py — a função geradora de sessão
from typing import Generator
from sqlalchemy.orm import Session

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db          # Entrega a sessão para o router
    finally:
        db.close()       # Fecha SEMPRE, mesmo se der erro

```

```

# Como usar em qualquer router
from fastapi import Depends
from sqlalchemy.orm import Session
from app.database import get_db

@router.get("/clientes/{id}", response_model=ClienteResponse)
def buscar_cliente(id: int, db: Session = Depends(get_db)):
    # db é a sessão do banco — válida só nesta requisição
    cliente = db.query(Cliente).filter(Cliente.id == id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado")
    return cliente

```

Por que yield ao invés de return ? Porque yield cria um *gerador* — o código depois do yield (o finally: db.close()) roda **após** a requisição terminar. É como um try/finally que envolve toda a requisição automaticamente.

Analogia VBA: Em VBA você abria a conexão no início da Sub e fechava no final. Aqui o FastAPI faz isso por você, garantindo que a conexão seja fechada mesmo se ocorrer um erro no meio.

CRUD completo — exemplo com Clientes

```
# app/routers/clientes.py
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from sqlalchemy.exc import IntegrityError
from typing import List

from app.database import get_db
from app.models.cliente import Cliente
from app.schemas.cliente import ClienteCreate, ClienteUpdate, ClienteResponse

router = APIRouter()

# GET lista – todos os clientes
@router.get("/", response_model=List[ClienteResponse])
def listar_clientes(db: Session = Depends(get_db)):
    return db.query(Cliente).order_by(Cliente.nome).all()

# GET por ID – um cliente específico
@router.get("/{id}", response_model=ClienteResponse)
def buscar_cliente(id: int, db: Session = Depends(get_db)):
    cliente = db.query(Cliente).filter(Cliente.id == id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado")
    return cliente

# POST – criar novo cliente
@router.post("/", response_model=ClienteResponse, status_code=201)
def criar_cliente(dados: ClienteCreate, db: Session = Depends(get_db)):
    # Verifica CPF/CNPJ duplicado (só quando preenchido)
    if dados.cpf_cnpj:
        existente = db.query(Cliente).filter(
            Cliente.cpf_cnpj == dados.cpf_cnpj
        ).first()
        if existente:
            raise HTTPException(
                status_code=400,
                detail=f"CPF/CNPJ {dados.cpf_cnpj} já cadastrado"
            )

    cliente = Cliente(**dados.model_dump())
    db.add(cliente)
    db.commit()
    db.refresh(cliente) # Recarrega do banco (pega o ID gerado)
```

```

    return cliente

# PATCH – atualização parcial
@router.patch("/{id}", response_model=ClienteResponse)
def atualizar_cliente(id: int, dados: ClienteUpdate, db: Session = Depends(get_db)):
    cliente = db.query(Cliente).filter(Cliente.id == id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado")

    # Atualiza só os campos que foram enviados (não None)
    for campo, valor in dados.model_dump(exclude_unset=True).items():
        setattr(cliente, campo, valor)

    db.commit()
    db.refresh(cliente)
    return cliente

# DELETE – com proteção contra exclusão de clientes com vendas
@router.delete("/{id}", status_code=204)
def deletar_cliente(id: int, db: Session = Depends(get_db)):
    cliente = db.query(Cliente).filter(Cliente.id == id).first()
    if not cliente:
        raise HTTPException(status_code=404, detail="Cliente não encontrado")

    # Verifica se tem vendas (soft protection)
    if cliente.vendas:
        raise HTTPException(
            status_code=400,
            detail="Não é possível excluir cliente com vendas cadastradas"
        )

    db.delete(cliente)
    db.commit()
    return None # 204 No Content

```

Endpoints especiais — PATCH /vendas/{id}/aprovar

Alguns endpoints não são CRUD simples — eles representam ações de negócio:

```

# app/routers/vendas.py
@router.patch("/{id}/aprovar", response_model=VendaResponse)
def aprovar_venda(id: int, db: Session = Depends(get_db)):
    """
    Aprovar uma venda é uma operação complexa:
    1. Muda o status para 'Aprovado'
    2. Registra a data de aprovação
    3. Gera lançamento no financeiro
    4. Gera prestação de contas para o colaborador responsável
    """

```

Toda essa lógica fica no SERVICE, não aqui.

"""

```
from app.services.venda_service import aprovar_venda_service
return aprovar_venda_service(db, id)
```

O Swagger UI — seu melhor amigo

Quando o backend está rodando, acesse:

<http://localhost:8000/docs>

Você verá uma interface interativa onde pode:

- Ver todos os endpoints organizados por tag
- Testar chamadas HTTP diretamente no navegador
- Ver os schemas de entrada e saída
- Ver os possíveis códigos de erro

É o equivalente a testar uma função VBA diretamente no Immediate Window do VBA Editor — mas para toda a API, com documentação automática gerada a partir do seu código.

Ponto-chave: Routers organizam endpoints por domínio. `Depends(get_db)` garante que cada requisição tem sua própria sessão de banco, fechada automaticamente. `exclude_unset=True` no PATCH é o que permite atualizações parciais sem sobrescrever campos não enviados. Endpoints como `/aprovar` representam transições de estado — delegar para o service, não implementar no router.

7. Camada de Serviços (Services)

Por que a lógica de negócio não deve ficar no router

O router tem uma responsabilidade: receber a requisição HTTP e devolver a resposta HTTP. Só isso.

Quando você coloca lógica de negócio no router, o arquivo cresce, fica difícil de testar, e a mesma lógica precisa ser duplicada se chamada de outro lugar.

Regra prática: se a operação envolve mais de uma tabela ou tem mais de 3 passos, crie um service.

Aprovar uma venda — o exemplo mais rico do projeto

```
# app/services/venda_service.py
from datetime import date
from sqlalchemy.orm import Session
```

```
from fastapi import HTTPException

from app.models.venda import Venda
from app.models.financeiro import Financeiro
from app.models.prestacao_contas import PrestacaoContas
from app.models.enums import StatusVenda, TipoLancamento, CategoriaFinanceiro
from app.models.enums import StatusPagamento, StatusPrestacao


def aprovar_venda_service(db: Session, venda_id: int) -> Venda:
    """
    Aprovar uma venda executa as seguintes operações ATOMICAMENTE:
    1. Valida que a venda existe e está em status aprovável
    2. Atualiza status e data de aprovação
    3. Cria lançamento financeiro (Receita)
    4. Gera prestação de contas para o colaborador responsável (se houver)
    """

    # --- 1. Busca e valida ---
    venda = db.query(Venda).filter(Venda.id == venda_id).first()
    if not venda:
        raise HTTPException(status_code=404, detail="Venda não encontrada")

    if venda.status_venda == StatusVenda.aprovado:
        raise HTTPException(status_code=400, detail="Venda já está aprovada")

    if venda.status_venda == StatusVenda.cancelado:
        raise HTTPException(status_code=400, detail="Não é possível aprovar venda cancelada")

    try:
        # --- 2. Atualiza a venda ---
        venda.status_venda = StatusVenda.aprovado
        venda.data_aprovacao = date.today()

        # --- 3. Cria lançamento financeiro ---
        lancamento = Financeiro(
            tipo = TipoLancamento.receita,
            categoria = CategoriaFinanceiro.venda_equipamentos,
            descricao = f"Venda #{venda.id} - {venda.cliente.nome}",
            valor = venda.valor_final,
            data_vencimento = date.today(),
            status_pagamento = StatusPagamento.pendente,
            forma_pagamento = venda.forma_pagamento,
            id_venda = venda.id
        )
        db.add(lancamento)

        # --- 4. Gera prestação de contas (se tiver responsável) ---
        if venda.id_responsavel and venda.responsavel:
            colab = venda.responsavel
            prestacao_id = _gerar_id_prestacao(db)

            prestacao = PrestacaoContas(
```

```

        id_prestacao      = prestacao_id,
        data              = date.today(),
        id_venda          = venda.id,
        id_colaborador    = colab.id_colaborador,
        valor_recebido    = venda.valor_final,
        custo_instalacao  = float(venda.valor_instalacao or 0),
        percentual_comissao = float(colab.percentual_comissao),
        status_pagto      = StatusPrestacao.pendente
    )
    db.add(prestacao)

    # --- COMMIT ATÔMICO: tudo ou nada ---
    db.commit()
    db.refresh(venda)
    return venda

except Exception as e:
    db.rollback() # Desfaz tudo se qualquer passo falhar
    raise HTTPException(status_code=500, detail=f"Erro ao aprovar venda: {str(e)}")

def _gerar_id_prestacao(db: Session) -> str:
    """Gera o próximo ID de prestação no formato 'PRE0001'"""
    from app.models.prestacao_contas import PrestacaoContas
    ultimo = db.query(PrestacaoContas).order_by(
        PrestacaoContas.id_prestacao.desc()
    ).first()
    if not ultimo:
        return "PRE0001"
    numero = int(ultimo.id_prestacao[3:]) + 1
    return f"PRE{numero:04d}"

```

Pagar prestação — gera despesa de comissão

```

# app/services/financeiro_service.py
def pagar_prestacao_service(db: Session, prestacao_id: str) -> PrestacaoContas:
    """
    Marcar uma prestação como paga:
    1. Valida que existe e está pendente
    2. Atualiza status e data de pagamento
    3. Cria lançamento de DESPESA de comissão no financeiro
    """
    prestacao = db.query(PrestacaoContas).filter(
        PrestacaoContas.id_prestacao == prestacao_id
    ).first()

    if not prestacao:
        raise HTTPException(status_code=404, detail="Prestação não encontrada")

    if prestacao.status_pagto != StatusPrestacao.pendente:
        raise HTTPException(status_code=400, detail="Prestação não está pendente")

```

```

try:
    # Calcula o valor da comissão
    valor_comissao = float(prestacao.valor_recebido) * float(prestacao.percentual_comissao)

    # Atualiza a prestação
    prestacao.status_pagto = StatusPrestacao.paga
    prestacao.data_pagto = date.today()

    # Cria despesa de comissão
    despesa = Financeiro(
        tipo = TipoLancamento.despesa,
        categoria = CategoriaFinanceiro.comissao,
        descricao = f"Comissão {prestacao.colaborador.nome} - Venda #{prestacao.id_v
        valor = valor_comissao,
        data_vencimento = date.today(),
        data_pagamento = date.today(),
        status_pagamento = StatusPagamento.pago,
        forma_pagamento = FormaPagamento.pix
    )
    db.add(despesa)

    db.commit()
    db.refresh(prestacao)
    return prestacao

except Exception as e:
    db.rollback()
    raise HTTPException(status_code=500, detail=f"Erro ao pagar prestação: {str(e)}")

```

Transações atômicas — o que são e por que importam

Atômico significa "tudo ou nada". Se a aprovação da venda criar o lançamento financeiro mas falhar ao criar a prestação, o `db.rollback()` desfaz **todas** as operações do bloco, voltando ao estado anterior.

Sem isso, você poderia ter uma venda "aprovada" sem lançamento financeiro, ou um lançamento sem a prestação correspondente — dados inconsistentes que causam problemas em cascata.

Analogia VBA: É como colocar várias operações dentro de um `On Error GoTo Rollback + Connection.BeginTrans/CommitTrans/RollbackTrans`. Você já fez isso para garantir integridade em Access — aqui é o mesmo conceito, com sintaxe diferente.

Ponto-chave: Services são onde a lógica de negócio vive. O padrão é: validar → executar passos → commit ou rollback. O `try/except` com `db.rollback()` garante que operações multi-tabela são atômicas — ou tudo é salvo, ou nada é. Isso protege a integridade dos dados.

8. Autenticação Simples

Como o login funciona neste projeto

```
# app/routers/usuarios.py
import bcrypt

@router.post("/login")
def login(dados: LoginSchema, db: Session = Depends(get_db)):
    usuario = db.query(Usuario).filter(Usuario.login == dados.login).first()

    if not usuario:
        raise HTTPException(status_code=401, detail="Credenciais inválidas")

    # Verifica a senha contra o hash armazenado
    senha_correta = bcrypt.checkpw(
        dados.senha.encode("utf-8"),
        usuario.senha_hash.encode("utf-8")
    )

    if not senha_correta:
        raise HTTPException(status_code=401, detail="Credenciais inválidas")

    # Retorna o objeto do usuário (sem a senha!)
    return UsuarioResponse.model_validate(usuario)
```

Como criar o hash da senha

```
# Ao criar usuário
senha_hash = bcrypt.hashpw(senha_texto.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
```

O bcrypt é um algoritmo de hash **lento por design** — isso dificulta ataques de força bruta. Você **nunca** guarda a senha em texto plano, apenas o hash.

Analogia VBA: Em sistemas VBA básicos, era comum guardar a senha em texto (ou em Base64, que não é criptografia). O bcrypt é o equivalente seguro — a senha não pode ser recuperada a partir do hash.

Sessão no Streamlit com `session_state`

O Streamlit não tem conceito nativo de "sessão persistente" como cookies do navegador. Usa-se `st.session_state` — um dicionário que persiste enquanto a aba do navegador estiver aberta:

```
# frontend/app.py
import streamlit as st
import requests
```

```
# Verifica se está logado
if "usuario" not in st.session_state:
    st.session_state.usuario = None

def fazer_login(login: str, senha: str):
    resp = requests.post(
        f"{API_URL}/usuarios/login",
        json={"login": login, "senha": senha}
    )
    if resp.status_code == 200:
        st.session_state.usuario = resp.json() # Guarda o objeto do usuário
        st.rerun() # Recarrega a página logado
    else:
        st.error("Login ou senha incorretos")

# Em qualquer página, verifica acesso
def verificar_acesso_admin():
    if not st.session_state.get("usuario"):
        st.error("Faça login primeiro")
        st.stop()
    if st.session_state.usuario["perfil"] != "Admin":
        st.error("Acesso restrito a administradores")
        st.stop()
```

Quando isso é suficiente e quando não é

É suficiente quando:

- Sistema interno de empresa (LAN ou VPN)
- Poucos usuários conhecidos
- Não há dados altamente sensíveis que exijam rastreabilidade

Não é suficiente quando:

- Sistema exposto na internet para usuários não-gerenciados
- Precisa de expiração automática de sessão (token JWT com TTL)
- Múltiplos serviços precisam validar o token (microservices)
- Auditoria de quem fez o quê e quando

Para esses casos, a solução correta é **JWT (JSON Web Token)**: o backend gera um token assinado criptograficamente com prazo de validade, e o frontend envia esse token em cada requisição no header `Authorization: Bearer <token>`.

Ponto-chave: Login com `bcrypt` + `session_state` é simples e suficiente para sistemas internos. A senha nunca é guardada em texto — só o hash. O `session_state` do Streamlit substitui cookies/JWT, mas não tem expiração automática e some ao fechar a aba. Para sistemas enterprise, use JWT com FastAPI OAuth2PasswordBearer.

9. Streamlit como Frontend

O que é Streamlit e por que foi escolhido

Streamlit é um framework Python que transforma scripts em aplicações web interativas. Você escreve Python e ele gera HTML/CSS/JavaScript por baixo dos panos.

Por que foi escolhido aqui:

- Time é de desenvolvedores Python, não front-end
- Velocidade de desenvolvimento: uma tela de CRUD com filtros em 50 linhas
- Ótimo para dashboards, visualizações, formulários internos
- Deploy gratuito no Streamlit Cloud

Limitação principal: Streamlit re-executa o script **inteiro** a cada interação do usuário. Cada clique, cada digitação em um campo, recomeça do zero. Isso é intuitivo no início, mas requer cuidado com chamadas de API caras.

Estrutura de páginas múltiplas

O Streamlit detecta automaticamente arquivos na pasta `pages/`. O prefixo numérico define a ordem no menu lateral:

```
frontend/  
├─ app.py           → página principal (login + dashboard)  
└─ pages/  
    ├─ 01_Clientes.py → /Clientes  
    ├─ 02_Vendas.py   → /Vendas  
    └─ ...
```

Como se comunica com o backend

```
# frontend/utils/api.py  
import requests  
import streamlit as st  
  
API_URL = "https://mvenergiasolar-production.up.railway.app"  
  
def get_clientes():  
    try:  
        resp = requests.get(f"{API_URL}/clientes/", timeout=10)  
        resp.raise_for_status() # Lança exceção se status >= 400  
        return resp.json()  
    except requests.exceptions.RequestException as e:  
        st.error(f"Erro ao conectar com o servidor: {e}")
```

```

        return []

def criar_cliente(dados: dict):
    resp = requests.post(f"{API_URL}/clientes/", json=dados, timeout=10)
    if resp.status_code == 201:
        return resp.json(), None
    else:
        return None, resp.json().get("detail", "Erro desconhecido")

```

Exemplo de página: Clientes

```

# frontend/pages/01_Clientes.py
import streamlit as st
from utils.api import get_clientes, criar_cliente, deletar_cliente

# Proteção de acesso
if "usuario" not in st.session_state or not st.session_state.usuario:
    st.warning("Faça login primeiro")
    st.stop()

st.title("👤 Clientes")

# Carrega dados (com cache de 60 segundos para não sobrecarregar a API)
@st.cache_data(ttl=60)
def carregar_clientes():
    return get_clientes()

clientes = carregar_clientes()

# Filtros
col1, col2, col3 = st.columns(3)
with col1:
    busca = st.text_input("Buscar por nome")
with col2:
    status_filtro = st.selectbox("Status", ["Todos", "Ativo", "Inativo", "Prospecto"])
with col3:
    cidade_filtro = st.text_input("Cidade")

# Aplica filtros localmente (não refaz a chamada de API)
resultado = clientes
if busca:
    resultado = [c for c in resultado if busca.lower() in c["nome"].lower()]
if status_filtro != "Todos":
    resultado = [c for c in resultado if c["status"] == status_filtro]
if cidade_filtro:
    resultado = [c for c in resultado if cidade_filtro.lower() in (c.get("cidade") or "").lower()]

# Exibe tabela
st.dataframe(resultado, use_container_width=True)

# Formulário de cadastro

```

```
with st.expander("+ Novo Cliente"):
    with st.form("form_novo_cliente"):
        nome = st.text_input("Nome *")
        tipo = st.selectbox("Tipo", ["PF", "PJ"])
        cpf = st.text_input("CPF/CNPJ")
        email = st.text_input("E-mail")

    if st.form_submit_button("Salvar"):
        if not nome:
            st.error("Nome é obrigatório")
        else:
            dados = {"nome": nome, "tipo_pessoa": tipo, "cpf_cnpj": cpf or None,
                    "email": email or None, "data_cadastro": str(date.today())}
            cliente, erro = criar_cliente(dados)
            if cliente:
                st.success("Cliente cadastrado!")
                st.cache_data.clear() # Limpa o cache para recarregar
                st.rerun()
            else:
                st.error(f"Erro: {erro}")
```

Limitações e quando considerar React/Next.js

Situação	Streamlit é suficiente?
Dashboard interno com 10-50 usuários	✅ Sim
Formulários e CRUD simples	✅ Sim
Relatórios e visualizações	✅ Sim
Interface pública com 1000+ usuários	❌ Não — use React
UX sofisticada (drag-and-drop, modais complexos)	❌ Não
PWA / Aplicativo mobile nativo	❌ Não
Atualizações em tempo real (WebSocket)	⚠️ Limitado

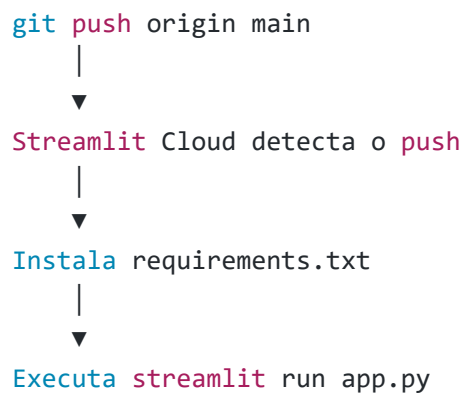
Ponto-chave: Streamlit é Python puro virando web app — ótimo para ferramentas internas e dashboards. Use `@st.cache_data` para não reprocessar dados a cada clique. Para sistemas públicos ou com UX complexa, o backend FastAPI permanece o mesmo — só troca o frontend por React/Next.js, que consome a mesma API.

10. Deploy na Railway e Streamlit Cloud

O que acontece no push para main



Para o Streamlit Cloud, o processo é similar mas mais simples:



Variáveis de ambiente

Credenciais **nunca** ficam no código. Elas ficam nas variáveis de ambiente:

```

# .env (desenvolvimento local – NÃO versionar, adicione ao .gitignore)
DATABASE_URL=sqlite:///./solar.db

# Railway (configurar no painel Railway → Variables)
DATABASE_URL=postgresql://user:senha@host:5432/dbname
SECRET_KEY=alguma_chave_aleatoria_longa

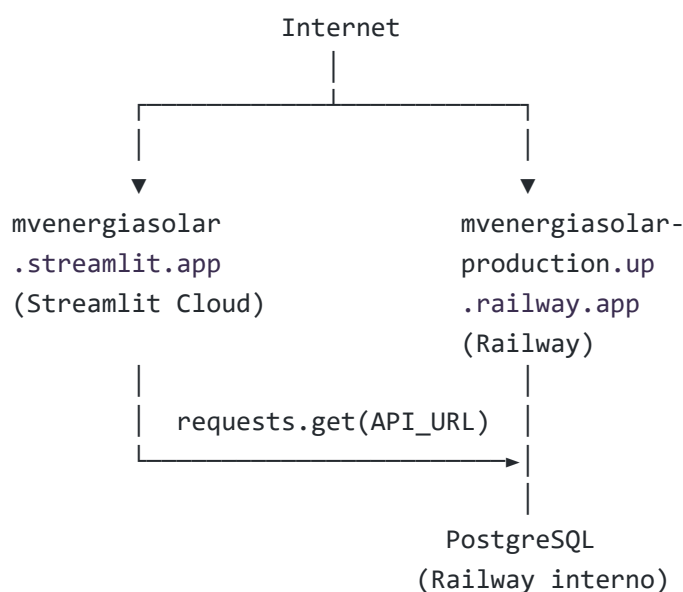
# Streamlit Cloud (configurar em Settings → Secrets)
API_URL=https://mvenergiasolar-production.up.railway.app

```

```
# No código, acessa com:
import os
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./solar.db")
API_URL = os.getenv("API_URL", "http://localhost:8000")

# No Streamlit, pode usar st.secrets (arquivo .streamlit/secrets.toml local)
# ou variáveis de ambiente normais:
API_URL = st.secrets.get("API_URL", os.getenv("API_URL", "http://localhost:8000"))
```

Por que backend e frontend ficam separados



- **Streamlit Cloud** tem plano gratuito, é ótimo para frontend Python, mas não roda servidores backend
- **Railway** roda qualquer container Docker, tem banco PostgreSQL integrado, mas cobra por uso
- A comunicação entre eles é HTTPS normal — o Streamlit "liga" para o FastAPI como qualquer cliente web

Analogia VBA: É como ter o formulário Excel em um computador e o banco Access em um servidor de rede — eles se comunicam pelo caminho de rede, não estão no mesmo arquivo.

Ponto-chave: O deploy na Railway é um pipeline automático: push no Git → build → migração → servidor. As variáveis de ambiente são o mecanismo para que o mesmo código funcione em desenvolvimento (SQLite) e produção (PostgreSQL) sem mudança. O `alembic upgrade head` no start command é o guardião da integridade do banco a cada deploy.

11. Boas Práticas que este Projeto Aplica

1. Separação de responsabilidades na prática

Cada camada faz **uma coisa**:

```
Router      → recebe HTTP, devolve HTTP, nada mais
Schema      → valida e serializa dados
Service     → implementa a lógica de negócio
Model       → define estrutura do banco
Database    → gerencia conexões
```

Se você se perguntar "onde fica este código?" e a resposta depender de "o que ele faz", você tem uma boa separação.

2. PATCH vs PUT — atualização parcial

Este projeto usa `PATCH` (não `PUT`) para atualizações. A diferença:

- `PUT` : substitui o recurso inteiro. Você precisa enviar todos os campos.
- `PATCH` : atualiza apenas os campos enviados.

O `exclude_unset=True` do Pydantic é o que torna isso possível:

```
# Pydantic distingue "campo não enviado" de "campo enviado como None"
dados = ClienteUpdate(nome="João") # só nome foi enviado
dados.model_dump(exclude_unset=True) # {"nome": "João"} – email, telefone etc. não aparecem
```

3. Enums ao invés de strings livres

```
# ❌ Ruim – String livre, qualquer valor é aceito
status = Column(String(20))
# O banco pode ter: "ativo", "Ativo", "ATIVO", "atv", "1"

# ✅ Bom – Enum, só valores definidos são aceitos
status = Column(SAEnum(StatusCliente))
# O banco só aceita: "Ativo", "Inativo", "Prospecto", "Bloqueado"
```

O banco PostgreSQL rejeita qualquer valor fora do enum em nível de banco — proteção dupla.

4. Soft delete vs hard delete

O projeto usa hard delete (exclusão real), mas com **proteção por integridade referencial**:

```
# Antes de deletar, verifica dependências
if cliente.vendas:
```

```
raise HTTPException(400, "Não é possível excluir cliente com vendas")
```

Em sistemas enterprise, usaria-se soft delete:

```
# Coluna adicionada ao model
deleted_at = Column(DateTime, nullable=True) # None = ativo, data = deletado

# Ao deletar
cliente.deleted_at = datetime.now()

# Em todas as queries
db.query(Cliente).filter(Cliente.deleted_at == None).all()
```

Soft delete permite recuperar dados, manter histórico e não quebrar referências.

5. Configuração via .env

```
# .gitignore – CRÍTICO
.env
solar.db
__pycache__/
*.pyc
.venv/

# .env.example – versionar como template
DATABASE_URL=sqlite:///./solar.db
SECRET_KEY=mude-isso-em-producao
```

Nunca escreva uma senha, URL de banco ou chave secreta diretamente no código. Qualquer pessoa com acesso ao repositório teria acesso ao banco de produção.

6. Tratamento de erros consistente

```
# Padrões de HTTP status code:
# 200 OK – GET bem sucedido
# 201 Created – POST bem sucedido
# 204 No Content – DELETE bem sucedido
# 400 Bad Request – Dado inválido, regra de negócio violada
# 401 Unauthorized – Não autenticado
# 403 Forbidden – Autenticado mas sem permissão
# 404 Not Found – Recurso não encontrado
# 422 Unprocessable Entity – Validação Pydantic falhou (FastAPI faz automático)
# 500 Internal Server Error – Erro inesperado do servidor

# Nunca retorne 200 com {"error": "..."} – use os status codes corretos
raise HTTPException(status_code=404, detail="Cliente não encontrado")
```

Ponto-chave: Boas práticas não são ornamentais — são o que torna o sistema mantível em 1 ano. PATCH com `exclude_unset` evita bugs de sobrescrita. Enums evitam dados inválidos no banco. Status HTTP corretos tornam o debug e o frontend previsíveis. `.env` evita vazamento de credenciais.

12. Pontos de Atenção e o que Fazer Diferente

O que este projeto faz de forma simplificada

Autenticação sem JWT

Como está: O backend retorna o objeto do usuário e o Streamlit guarda em `session_state`. Se o usuário abre a mesma URL em outra aba, precisa logar de novo. Não há expiração automática.

Como seria em produção:

```
# JWT com expiração de 8 horas
from jose import jwt
from datetime import datetime, timedelta

def criar_token(usuario_id: int, perfil: str) -> str:
    payload = {
        "sub": str(usuario_id),
        "perfil": perfil,
        "exp": datetime.utcnow() + timedelta(hours=8)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

Sem testes automatizados

Como está: Você testa manualmente no Swagger UI ou no Streamlit.

Como deveria ser:

```
# tests/test_clientes.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_criar_cliente():
    resp = client.post("/clientes/", json={
        "nome": "João Silva",
        "tipo_pessoa": "PF",
        "data_cadastro": "2024-01-15"
    })
```

```

assert resp.status_code == 201
assert resp.json()["nome"] == "João Silva"

def test_cpf_duplicado():
    dados = {"nome": "A", "cpf_cnpj": "12345678901", "data_cadastro": "2024-01-15"}
    client.post("/clientes/", json=dados)
    resp = client.post("/clientes/", json=dados)
    assert resp.status_code == 400

```

Rode com `pytest tests/` a cada commit. Isso evita que mudanças quebrem funcionalidades existentes.

Sem logs estruturados

Como está: Erros podem aparecer nos logs do Railway, mas não há rastreabilidade.

Como seria:

```

import logging
import json

logger = logging.getLogger(__name__)

# Em vez de print ou deixar passar silencioso:
logger.info(json.dumps({
    "evento": "venda_aprovada",
    "venda_id": venda_id,
    "usuario": usuario_id,
    "timestamp": datetime.utcnow().isoformat()
}))

```

Sem rate limiting

Como está: Qualquer pessoa pode chamar `/usuarios/login` 10.000 vezes por segundo — ataque de força bruta.

Como seria:

```

from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@router.post("/login")
@limiter.limit("5/minute") # Máximo 5 tentativas por minuto por IP
def login(request: Request, dados: LoginSchema, db: Session = Depends(get_db)):
    ...

```

O que adicionar se o sistema crescer

Necessidade	Solução
Testes automatizados	pytest + pytest-asyncio + banco SQLite em memória
Autenticação robusta	FastAPI OAuth2 + JWT com python-jose
Logs estruturados	structlog OU loguru
Rate limiting	slowapi
Cache de queries pesadas	Redis com redis-py OU aioredis
Tarefas em background (emails, PDFs)	Celery + Redis como broker
CI/CD	GitHub Actions: roda testes a cada PR, deploy automático se passar
Monitoramento de erros	Sentry (gratuito para projetos pequenos)
Documentação da API	Já tem! O Swagger (/docs) é gerado pelo FastAPI

Como pensar em escalabilidade sem over-engineering

A regra de ouro é: **resolva o problema real que você tem hoje, mas não feche portas para o futuro.**

Este projeto faz isso bem:

- O FastAPI pode receber JWT amanhã sem mudar os routers
- O SQLAlchemy troca de SQLite para PostgreSQL sem mudar os models
- O Streamlit pode ser substituído por React sem mudar uma linha do backend

O que **não** vale a pena fazer no início:

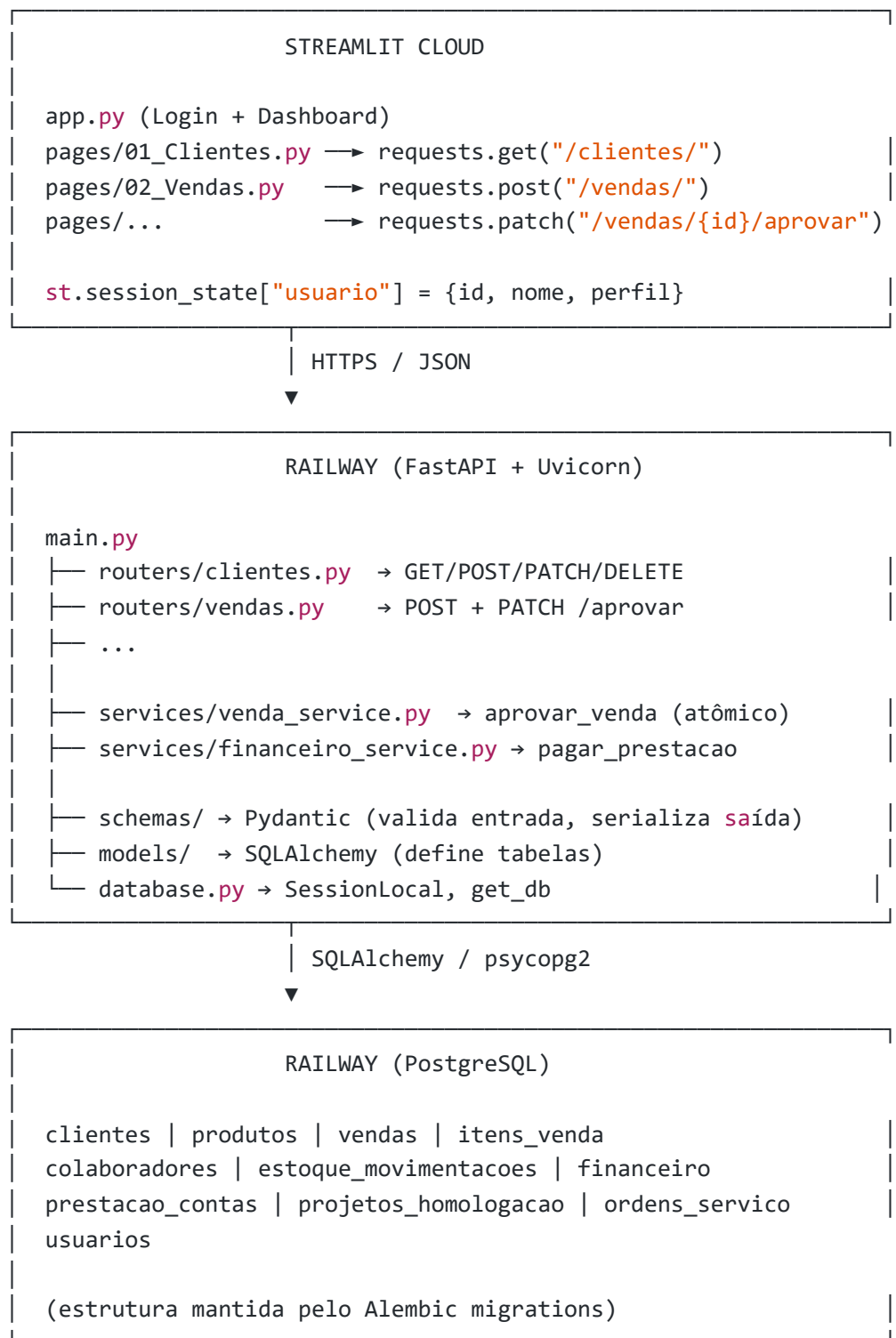
- Microservices (divisão em múltiplos serviços separados) — complexidade enorme, benefício zero para um sistema pequeno
- Cache com Redis — só quando queries ficarem lentas medidas com dados reais
- Filas assíncronas — só quando houver operações que demoram mais de 5 segundos

A sequência correta é:

1. **Faça funcionar** (como este projeto faz)
2. **Faça funcionar corretamente** (adicione testes)
3. **Faça funcionar bem** (otimize onde mede lentidão real)
4. **Faça funcionar de forma segura** (adicione JWT, rate limiting, auditoria)

Ponto-chave: Este projeto é um excelente ponto de partida — produção real, código organizado, práticas sólidas. As simplificações (sem JWT, sem testes) são escolhas conscientes de velocidade vs. robustez. Para crescer, adicione testes primeiro (te dão confiança para mudar código), JWT depois (segurança), e só então pense em cache, filas e microservices quando os dados provarem que você precisa.

Resumo Visual da Arquitetura Completa



Guia gerado com base na especificação completa do projeto MV Energia Solar v1.0 Para dúvidas específicas sobre implementação, consulte cada seção com os exemplos de código acima.